

Algorithms & Data Structures

Assignment – Complete Answers

Exercise 1: Inventory Management System

1. Why Data Structures & Algorithms Are Essential

Large inventories may contain thousands of products. Without efficient data structures, operations like searching, updating, and deleting a product would require scanning every record ($O(n)$). Proper data structures reduce this to $O(1)$ or $O(\log n)$, making the system fast and scalable.

2. Suitable Data Structures

- **HashMap (Dictionary)** – $O(1)$ average-case for add, update, delete, and lookup by productId.
- **ArrayList** – $O(1)$ for append; $O(n)$ for search/delete. Good for ordered traversal.
- **TreeMap** – $O(\log n)$ operations; maintains sorted order by key.

For this scenario, **HashMap<Integer, Product>** keyed on productId is the optimal choice.

3. Implementation (Java)

```
class Product {
    int productId;
    String productName;
    int quantity;
    double price;

    public Product(int id, String name, int qty, double price) {
        this.productId = id;
        this.productName = name;
        this.quantity = qty;
        this.price = price;
    }
}

import java.util.HashMap;

class Inventory {
    private HashMap<Integer, Product> inventory = new HashMap<>();

    // Add product - O(1)
    public void addProduct(Product p) {
        inventory.put(p.productId, p);
    }
}
```

```
// Update product - O(1)
public void updateProduct(int id, int newQty, double newPrice) {
    if (inventory.containsKey(id)) {
```

```

        inventory.get(id).quantity = newQty;
        inventory.get(id).price    = newPrice;
    }
}

// Delete product - O(1)
public void deleteProduct(int id) {
    inventory.remove(id);
}

// Search - O(1)
public Product getProduct(int id) {
    return inventory.get(id);
}
}

```

4. Time Complexity Analysis

Operation	Data Structure	Time Complexity
Add	HashMap	O(1) average
Update	HashMap	O(1) average
Delete	HashMap	O(1) average
Search	HashMap	O(1) average

Optimization: Use *initial capacity* in HashMap constructor to reduce rehashing. For sorted traversal, switch to TreeMap (O(log n)).

Exercise 2: E-Commerce Platform Search Function

1. Big O Notation Explained

Big O notation describes the **upper bound** of an algorithm's running time as the input size n grows. It expresses worst-case growth rate, ignoring constants.

- $O(1)$ – Constant time (HashMap lookup)
- $O(\log n)$ – Logarithmic (Binary Search)
- $O(n)$ – Linear (Linear Search)
- $O(n \log n)$ – Merge Sort, Quick Sort average
- $O(n^2)$ – Bubble Sort worst case

2. Implementation

```

class Product {
    int productId;
    String productName;
    String category;
}

```

```

    public Product(int id, String name, String cat) {
        this.productId = id;
        this.productName = name;
        this.category = cat;
    }
}

// Linear Search - O(n)
public static Product linearSearch(Product[] products, String name) {
    for (Product p : products) {
        if (p.productName.equalsIgnoreCase(name)) return p;
    }
    return null;
}

// Binary Search - O(log n) - array must be sorted by productId
public static Product binarySearch(Product[] products, int targetId) {
    int low = 0, high = products.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (products[mid].productId == targetId) return products[mid];
        else if (products[mid].productId < targetId) low = mid + 1;
        else high = mid - 1;
    }
    return null;
}

```

3. Comparison

Algorithm	Best Case	Average Case	Worst Case	Requires Sorted?
Linear Search	O(1)	O(n)	O(n)	No
Binary Search	O(1)	O(log n)	O(log n)	Yes

Recommendation: Binary Search is preferred for large, sorted product catalogs. For unsorted or frequently changing data, Linear Search or HashMap is better.

Exercise 3: Sorting Customer Orders

1. Sorting Algorithms Overview

- **Bubble Sort** – Repeatedly swaps adjacent elements. $O(n^2)$ worst case. Simple but slow.
- **Insertion Sort** – Builds sorted array one element at a time. $O(n^2)$ worst, $O(n)$ best.
- **Quick Sort** – Divide-and-conquer using pivot. $O(n \log n)$ average, $O(n^2)$ worst.
- **Merge Sort** – Divide-and-conquer, always $O(n \log n)$. Stable but uses extra $O(n)$ space.

2. Implementation

```

class Order {

```

```

    int orderId;
    String customerName;
    double totalPrice;

    public Order(int id, String name, double price) {
        this.orderId      = id;
        this.customerName = name;
        this.totalPrice    = price;
    }
}

// Bubble Sort - O(n^2)
public static void bubbleSort(Order[] orders) {
    int n = orders.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (orders[j].totalPrice > orders[j + 1].totalPrice) {
                Order temp      = orders[j];
                orders[j]       = orders[j + 1];
                orders[j + 1]    = temp;
            }
        }
    }
}

// Quick Sort - O(n log n) average
public static void quickSort(Order[] orders, int low, int high) {
    if (low < high) {
        int pi = partition(orders, low, high);
        quickSort(orders, low, pi - 1);
        quickSort(orders, pi + 1, high);
    }
}

private static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].totalPrice;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (orders[j].totalPrice <= pivot) {
            i++;
            Order temp = orders[i];
            orders[i]   = orders[j];
            orders[j]   = temp;
        }
    }
    Order temp      = orders[i + 1];
    orders[i + 1]    = orders[high];
    orders[high]     = temp;
    return i + 1;
}

```

3. Comparison

Bubble Sort makes $O(n^2)$ comparisons in all cases while Quick Sort averages $O(n \log n)$. For 1000 orders, Bubble Sort does ~500,000 comparisons vs Quick Sort's ~10,000. **Quick Sort is generally preferred** due

to its superior average-case performance and in-place sorting ($O(\log n)$ stack space). Merge Sort is preferred when stability is required.

Exercise 4: Employee Management System

1. Array Representation in Memory

Arrays store elements in **contiguous memory locations**. The base address plus index * element size gives direct $O(1)$ access to any element. Advantages include cache-friendly access patterns and minimal memory overhead.

2. Implementation

```
class Employee {
    int employeeId;
    String name;
    String position;
    double salary;

    public Employee(int id, String name, String pos, double sal) {
        this.employeeId = id;
        this.name        = name;
        this.position     = pos;
        this.salary       = sal;
    }
}

class EmployeeManager {
    private Employee[] employees;
    private int size = 0;

    public EmployeeManager(int capacity) {
        employees = new Employee[capacity];
    }

    // Add - O(1) if space available
    public void add(Employee e) {
        if (size < employees.length) employees[size++] = e;
    }

    // Search - O(n)
    public Employee search(int id) {
        for (int i = 0; i < size; i++)
            if (employees[i].employeeId == id) return employees[i];
        return null;
    }

    // Traverse - O(n)
    public void traverse() {
        for (int i = 0; i < size; i++)
            System.out.println(employees[i].name);
    }
}
```

```

    }

    // Delete - O(n) due to shifting
    public void delete(int id) {
        for (int i = 0; i < size; i++) {
            if (employees[i].employeeId == id) {
                for (int j = i; j < size - 1; j++)
                    employees[j] = employees[j + 1];
                employees[--size] = null;
                return;
            }
        }
    }
}

```

3. Time Complexity & Limitations

Operation	Time Complexity	Notes
Add	$O(1)$	Amortized when using dynamic array
Search	$O(n)$	Linear scan required
Traverse	$O(n)$	Visit every element
Delete	$O(n)$	Shifting required after removal

Limitations of arrays: Fixed size (Java), costly insertion/deletion in middle, no built-in resizing. Use arrays when size is known and random access is frequent.

Exercise 5: Task Management System

1. Types of Linked Lists

- **Singly Linked List** – Each node points to the next. Traversal is one-directional.
- **Doubly Linked List** – Each node has next and prev pointers. Supports bidirectional traversal.
- **Circular Linked List** – Tail points back to head. Useful for round-robin scheduling.

2. Implementation (Singly Linked List)

```

class Task {
    int taskId;
    String taskName;
    String status;
    Task next;

    public Task(int id, String name, String status) {
        this.taskId = id;
        this.taskName = name;
        this.status = status;
        this.next = null;
    }
}

```

```

    }
}

class TaskManager {
    private Task head = null;

    // Add at end - O(n)
    public void add(Task t) {
        if (head == null) { head = t; return; }
        Task curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = t;
    }

    // Search - O(n)
    public Task search(int id) {
        Task curr = head;
        while (curr != null) {
            if (curr.taskId == id) return curr;
            curr = curr.next;
        }
        return null;
    }

    // Traverse - O(n)
    public void traverse() {
        Task curr = head;
        while (curr != null) {
            System.out.println(curr.taskId + ": " + curr.taskName);
            curr = curr.next;
        }
    }

    // Delete - O(n)
    public void delete(int id) {
        if (head == null) return;
        if (head.taskId == id) { head = head.next; return; }
        Task prev = head;
        while (prev.next != null && prev.next.taskId != id)
            prev = prev.next;
        if (prev.next != null) prev.next = prev.next.next;
    }
}

```

3. Advantages Over Arrays

- Dynamic size – no pre-allocation needed.
- O(1) insertion/deletion at head (no shifting).
- Efficient for frequent insertions at arbitrary positions.

Disadvantage: O(n) random access (no index), extra memory for pointers.

Exercise 6: Library Management System

1. Search Algorithms

Linear Search: Scans each element sequentially until a match is found or the list ends. Works on unsorted data. $O(n)$.

Binary Search: Repeatedly halves the search space. Requires sorted data. $O(\log n)$.

2. Implementation

```
class Book {
    int bookId;
    String title;
    String author;

    public Book(int id, String title, String author) {
        this.bookId = id;
        this.title = title;
        this.author = author;
    }
}

// Linear Search by title - O(n)
public static Book linearSearch(Book[] books, String title) {
    for (Book b : books)
        if (b.title.equalsIgnoreCase(title)) return b;
    return null;
}

// Binary Search by title (sorted array) - O(log n)
public static Book binarySearch(Book[] books, String title) {
    int low = 0, high = books.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int cmp = books[mid].title.compareToIgnoreCase(title);
        if (cmp == 0) return books[mid];
        else if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return null;
}
```

3. When to Use Each

- **Linear Search:** Small datasets, unsorted data, single search.
- **Binary Search:** Large sorted datasets, frequent searches.

For a library with 10,000 books: Linear = up to 10,000 comparisons; Binary = at most 14 comparisons ($\log_2 10000 \approx 13.3$).

Exercise 7: Financial Forecasting

1. Recursion Explained

Recursion is a technique where a function calls itself with a smaller subproblem until a **base case** is reached. It simplifies problems that have self-similar structure (e.g., computing compound interest over n years).

2. Implementation

```
public class FinancialForecast {

    /**
     * Recursively calculates future value.
     *  $FV = PV * (1 + r)^n$ 
     * @param presentValue - Initial amount
     * @param growthRate - Annual growth rate (e.g., 0.08 for 8%)
     * @param years - Number of years to forecast
     * @return future value
     */
    public static double futureValue(double presentValue,
                                     double growthRate,
                                     int years) {

        // Base case
        if (years == 0) return presentValue;
        // Recursive case
        return futureValue(presentValue * (1 + growthRate),
                           growthRate, years - 1);
    }

    // Memoized (optimized) version
    private static java.util.HashMap<Integer, Double> memo = new java.util.HashMap<>();

    public static double futureValueMemo(double pv, double rate, int n) {
        if (n == 0) return pv;
        if (memo.containsKey(n)) return memo.get(n);
        double result = futureValueMemo(pv * (1 + rate), rate, n - 1);
        memo.put(n, result);
        return result;
    }

    public static void main(String[] args) {
        double fv = futureValue(10000, 0.08, 10);
        System.out.printf("Future Value: %.2f%n", fv); // ■21,589.25
    }
}
```

3. Complexity Analysis

- **Time Complexity:** $O(n)$ – one recursive call per year.
- **Space Complexity:** $O(n)$ – call stack depth equals number of years.
- **Optimization:** Use memoization or convert to iterative ($O(1)$ space).

Iterative alternative: `double fv = pv * Math.pow(1 + rate, n);` – O(1) time and space using Java's built-in `Math.pow`.